

Week 3 - Day 11a: Time Complexity Recap

Time complexity refers to the computational complexity that describes the number of operations the computer must perform to run an algorithm. It is expressed **as a function of the size of the input to the program**.

The most common runtimes, expressed in Big-O notation (ignoring constants and factors), are:

- $O(1)$ - Constant time
- $O(\log n)$ - Logarithmic time
- $O(n)$ - Linear time
- $O(n \log n)$ - Log-linear time
- $O(n^2)$ - Quadratic time
- $O(n^3)$ - Cubic time
- $O(2^n)$ - Exponential time
- $O(n!)$ - Factorial time

To determine the runtime of an algorithm, we typically count the number of operations the computer must perform as a function of the size of the input to the program and then convert the function to Big-O notation.

- $n \rightarrow O(n)$
- $n+5 \rightarrow O(n)$
- $n+200000 \rightarrow O(n)$
- $n-200000 \rightarrow O(n)$
- $2n \rightarrow O(n)$
- $n/2 \rightarrow O(n)$

Let's explore these concepts in more detail in the exercises below.

Exercise 1: Big O Notation []

For each of the following, give the number of operations in **big O notation**.

1.1 $1000000n+1000$ operations

1.2 $4n^2 - 10000n^{1.5} + 50$ operations

1.3 $n+0.1\sqrt{n}$ operations

1.4 $n^2 \log_3 n$ operations

1.5 $\log_2(n^3)$ operations

1.6 $n+\log_2 n$ operation

1.7 2^{n+1} operations

1.8 $2^n + n^2$ operations

Exercise 2: Analyzing Time Complexity [] []

What is the time complexity of the following functions in Big-O notation?

2.1 What is the time complexity of the `sum_numbers` function?

```
def sum_numbers(n):  
    summ = 0  
    for i in range(n):  
        summ += i  
    return summ
```

2.2 What is the time complexity of the `sum_even_numbers` function?

```
def sum_even_numbers(n):  
    summ = 0  
    for i in range(0, n, 2):  
        summ += i  
    return summ
```

2.3 What is the time complexity of the `sum_numbers2` function?

```
def sum_numbers2(n):  
    return n * (n-1) // 2
```

2.4 What is the time complexity of the `sum_and_prod` function?

```
def sum_and_prod(n):  
    summ = 0  
    for i in range(1, n):  
        summ += i  
    prod = 1  
    for i in range(1, n):  
        prod *= i  
    return [summ, prod]
```

2.5 What is the time complexity of the `sum_and_prod2` function?

```
def sum_and_prod2(n):  
    summ = sum_numbers(n) # calling the function from 2.1  
    prod = 1  
    for i in range(1, n):  
        prod *= i  
    return [summ, prod]
```

2.6 What is the time complexity of the `print_all_pairs` function?

```
def print_all_pairs(n):
    for i in range(n):
        for j in range(n):
            print(f"({i}, {j})")
```

2.7 What is the time complexity of the `collect_all_pairs` function?

```
def collect_all_pairs(n):
    pairs = []
    for i in range(n):
        new_pairs = []
        for j in range(n):
            new_pairs.append([i, j])
        pairs.append(new_pairs)
    return pairs
```

2.8 What is the time complexity of the `print_powers_up_to_n` function?

```
def print_powers_up_to_n(n):
    i = 1
    while i < n:
        print(i)
        i *= 2
```

2.9 What is the time complexity of the `find_sqrt_of_list` function?

```
def find_sqrt_of_list(n):
    for k in range(2, n + 1):
        j = 1
        while j * j <= k:
            j += 1
```

2.10 What is the time complexity of the `print_rectangle` function?

```
def print_rectangle(n):
    for i in range(n):
        for j in range(10):
            print('*', end='')
        print()
```

2.11 What is the time complexity of the `print_triangle` function?

```
def print_triangle(n):
    for i in range(n+1):
        for j in range(i+1):
```

```
print('*', end='')  
print()
```

Exercise 3: Time Complexity of pre-defined Functions [] []

Not all pre-defined functions take constant time.

What do you think is the **worst case** time complexity of the following functions in Big-O notation?

(Make a guess, we do not expect you to know the answer.)

3.1 What is the time complexity of finding the maximum element in a list of length n ?

```
x = max(lst)
```

3.2 What is the time complexity of accessing the first element of a list of length n ?

```
x = lst[0]
```

3.3 What is the time complexity of slicing a list from the second element to the end in a list of length n ?

```
x = lst[1:]
```

3.4 What is the time complexity of appending an element to the end of a list of length n ?

```
lst += [1]
```

3.5 What is the time complexity of creating a new list by concatenating an element to a list of length n ?

```
new_lst = lst + [1]
```

3.6 What is the time complexity of popping the last element from a list of length n ?

```
lst.pop() # This removes the last element
```

3.7 What is the time complexity of popping the 10th element from a list of length n ?

```
lst.pop(10) # This removes the 10th element
```

3.8 What is the time complexity of creating a list of squares of the first n integers?

```
[i**2 for i in range(n)]
```

3.9 What is the time complexity of checking if an element is in a list of length n ?

```
elem in lst # This tells us whether the element is in the list
```

Exercise 4: Time Complexity of Dictionary Operations []

Not all pre-defined functions take constant time.

What do you think is the **worst case** time complexity of the following functions in Big-O notation?

(Make a guess, we do not expect you to know the answer.)

4.1 What is the time complexity of accessing a value in a dictionary with n elements?

```
x = dct[key]
```

4.2 What is the time complexity of setting a value in a dictionary with n elements?

```
dct[key] = x
```

4.3 What is the time complexity of deleting a key-value pair in a dictionary with n elements?

```
del dct[key]
```

4.4 What is the time complexity of retrieving all keys from a dictionary with n elements?

```
dct.keys() # This iterates through every key and returns them as a list
```

4.5 What is the time complexity of checking if an element is in a dictionary with n elements?

```
elem in dct # This tells us whether the element is in the dictionary
```

Exercise 5: Time Complexity with Specific n []

What is n ? Sometimes, we need to specify.

5.1 What is the time complexity of the `find_min` function, where n is the length of `lst`?

```
def find_min(lst):  
    min_e = lst[0]  
    for e in lst[1:]:  
        if e < min_e:  
            min_e = e  
    return min_e
```

5.2 What is the time complexity of the `find_matrix_min` function, where `matrix` is a square $n \times n$ matrix?

```
def find_matrix_min(matrix):
    min_e = matrix[0][0]
    for row in matrix:
        min_row = find_min(row) # calling the function from 5.1
        if min_row < min_e:
            min_e = min_row
    return min_e
```

5.3 What is the time complexity of the `count_letter` function, where `n` is the length of the string?

```
def count_letter(string, letter):
    count = 0
    for c in string:
        if c == letter:
            count += 1
    return count
```

5.4 What is the time complexity of the `count_all_letters` function, where `n` is the length of the string?

```
def count_all_letters(string):
    counts = {}
    for letter in 'abcdefghijklmnopqrstuvwxyz':
        counts[letter] = count_letter(string, letter) # calling the function from 5.3
    return counts
```

5.5 What is the time complexity of the `count_all_letters2` function, where `n` is the length of the string?

```
def count_all_letters2(string):
    counts = {}
    for letter in string:
        if letter in counts:
            counts[letter] += 1
        else:
            counts[letter] = 1
    return counts
```

5.6 What is the time complexity of the `make_black` function, where `canvas` is an image of dimension `n*n`?

```
from simpleimage import SimpleImage

def make_black(canvas):
    for column in range(canvas.width):
```

```
for row in range(canvas.height):
    canvas.set_rgb(column, row, 0, 0, 0)
```

Exercise 6: Worst-Case and Best-Case Time Complexity

[] [] [] []

Sometimes, the number of operations **depends on the exact input**. In this case, we consider how many **operations** are needed in the **worst case**.

What is the **worst-case** time complexity of the following functions in Big-O notation?

Bonus: For which inputs do we need fewer operations? How many operations are needed in this case (best case)?

6.1 What is the worst-case time complexity of the `find_element` function, where `n` is the length of `lst`?

```
def find_element(lst, elem):
    for i in range(len(lst)):
        if lst[i] == elem:
            return i # position of the element
    return -1 # element not found
```

6.2 What is the worst-case time complexity of the `add_sums_list` function, where `n` is the length of `lst`, and the elements of the list are smaller than `n`?

```
def add_sums_list(lst):
    summ = 0
    for i in lst:
        summ += sum_numbers(i) # Using the function from Exercise 2.1
    return summ
```

6.3 What is the worst-case time complexity of the `add_sums_list` function, where `n` is the length of `lst`, and the elements of the list are smaller than `n`?

```
def add_sums_list(lst):
    summ = 0
    for i in lst:
        summ += sum_numbers2(i) # Using the function from Exercise
2.3
    return summ
```

6.4 What is the worst-case time complexity of the `draw_horizontal` function, where `canvas` is an image of dimension `n*n`?

```
from simpleimage import SimpleImage
```

```
def draw_horizontal(canvas, x, y, length):
    for i in range(length):
        canvas.set_rgb(x+i, y, 0, 0, 0)
```

Exercise 7: Worst-Case Time Complexity [] [] [] []

What is the **worst-case** time complexity of the following functions in Big-O notation?

7.1 What is the worst-case time complexity of the `factorial` function?

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n-1)
```

7.2 What is the worst-case time complexity of the `exp` function?

```
def exp(a, n):
    if n == 0:
        return 1
    else:
        return a * exp(a, n-1)
```

7.3 What is the worst-case time complexity of the `sum_all` function, where `lst` is a list of length `n`?

```
def sum_all(lst):
    if len(lst) == 0:
        return 0
    else:
        return lst[0] + sum_all(lst[1:])
```

7.4 What is the worst-case time complexity of the `rounded_log2` function?

```
def rounded_log2(n):
    if n <= 1:
        return 0
    return 1 + rounded_log2(n // 2)
```

Exercise 8: Finding Duplicates [] [] [] []

The following function finds whether there is an element that appears twice in a list and returns `True` if such an element is found.

Example:

```
["a", "b", "a", "c"] ---> True # because "a" appears twice
```

```
def has_repetition(lst):
    for i in range(len(lst)):
        for j in range(len(lst)):
            if i != j and lst[i] == lst[j]:
                return True
    return False

print(has_repetition(["a", "b", "a", "c"])) # True
print(has_repetition(["a", "b", "d", "c"])) # False
```

8.1 What is the **worst-case** time complexity of the above function?

8.2 Write a function that does the same but has a better worst-case time complexity.

Hint: Use a dictionary

```
# TODO: Your code here
def has_repetition_fast(lst):
    pass

# Test cases
print(has_repetition_fast(["a", "b", "a", "c"])) # True
print(has_repetition_fast(["a", "b", "d", "c"])) # False
```

8.3 What is the **worst-case** time complexity of your improved function?

Exercise 9: Sum of Two Numbers [] [] [] []

Given a list of n numbers and another number s , we want to know if there are two numbers in the list that sum up to s .

Example:

```
lst = [1, 3, 2, 6, 8]
s = 10
# This should return True, because 2 + 8 = 10.
```

9.1 Write a function `sum_to` that takes a list `lst` and an integer `s` and returns `True` if there are two numbers in `lst` summing up to `s`, and `False` otherwise.

Hint: Use a nested for loop.

```
# TODO: Your code here
def sum_to(lst, s):
    pass

# Use this code to test your code
assert sum_to([1, 3, 2, 6, 8], 10) == True # 2 + 8 = 10
assert sum_to([1, 5, 2, 6, 8], 4) == False
assert sum_to([1, 2, 3, 4, 5], 9) == True # 4 + 5 = 9
```

```

assert sum_to([1, 2, 3, 4, 5], 11) == False
assert sum_to([-1, 0, 1, 2], 1) == True # -1 + 2 = 1
assert sum_to([10, 15, 3, 7], 17) == True # 10 + 7 = 17
assert sum_to([3, 3], 6) == True # 3 + 3 = 6
assert sum_to([1], 2) == False # Single element list
assert sum_to([], 0) == False # Empty list

print("All tests passed. ☐")

```

9.2 What is the **worst-case** time complexity of your function?

9.3 Write a function that does the same but has a better worst-case time complexity!

Hint: Convert the list into a set before doing anything else.

```

# TODO: Your code here
def sum_to_fast(lst, s):
    pass

# Use this code to test your code
assert sum_to_fast([1, 3, 2, 6, 8], 10) == True # 2 + 8 = 10
assert sum_to_fast([1, 5, 2, 6, 8], 4) == False
assert sum_to_fast([1, 2, 3, 4, 5], 9) == True # 4 + 5 = 9
assert sum_to_fast([1, 2, 3, 4, 5], 11) == False
assert sum_to_fast([-1, 0, 1, 2], 1) == True # -1 + 2 = 1
assert sum_to_fast([10, 15, 3, 7], 17) == True # 10 + 7 = 17
assert sum_to_fast([3, 3], 6) == True # 3 + 3 = 6
assert sum_to_fast([1], 2) == False # Single element list
assert sum_to_fast([], 0) == False # Empty list

print("All tests passed. ☐")

```

9.4 What is the **worst-case** time complexity of your improved function?

Congratulations ☐